AI를 활용한 데이터 과학

- 실전 경험을 통한 머신러닝 응용 –

2026. Fall

조상구

내용

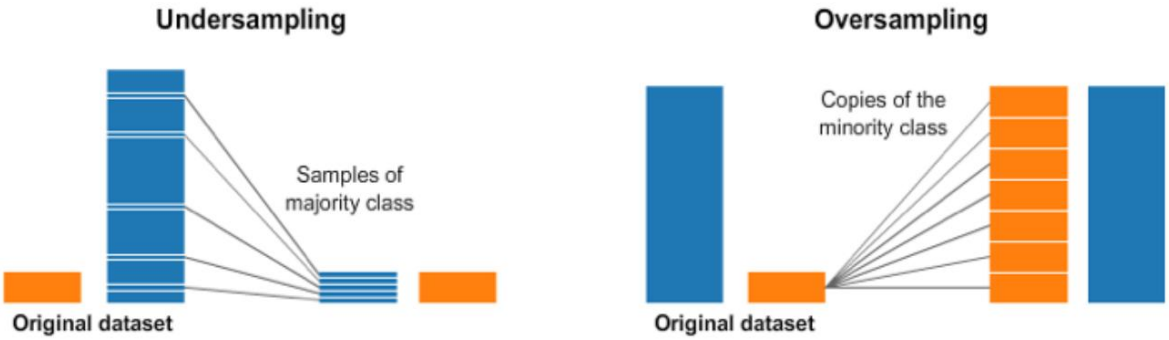
- 불균형 데이터 특징 3
- 다양한 성능 지표 적용 6
- 손실함수 조정 13
- Under-Over-Combine-ensemble classifier sampling 15
- Data downsizing 20

불균형 데이터(Imbalance data)

■ 불균형 데이터 특징

관심 대상이 되는 범주에 속하는 데이터의 비중이 10% 미만인 자료를 보통 불균형데이터(Imbalanced Data)라고 한다. 데이터의 불균형이 심할수록 예측 성능이 떨어지고 허위음성율이 높아지게 된다. 관심 대상이 되는 범주는 주로 소비자가 구매할 것인가? 환자가 질병을 앓고 있는가? 신용불량자인가? 등인데 모집단에서 차지하는 낮은 비중이 예측을 어렵게 한다. 불균형데이터에서는 모형의 정확도가 높아도 소수 범주 유형의 재현율(recall)이 낮아진다. 부도예측, 카드사고, 이탈고객, 스팸 메일, 의료진단 등의 분야는 허위양성율을 낮추고 재현율을 높여야 하는 이유이다.

데이터 불균형 문제의 해결방안으로 표본추출(Sampling)을 이용한 오버샘플링(Over-sampling)과 언더샘플링(Under-sampling) 방법이 있다(그림[]).



출처 : https://oralytics.com/2019/07/01/managing-imbalanced-data-sets-with-smote-in-python/

오버샘플링은 무작위로 소수의 데이터를 사전에 기준을 정해서 복제하는 방법으로 정보가 손실되지 않는다는 장점이 있으나, 여러 유형의 관측치를 다수 추가하여 과적합을 초래할 수 있는 단점이 있다. 반면에 언더샘플링의 경우 데이터의 손실이 크고, 때로는 중요한 자료를 버리는 위험이 존재한다. 데이터의 특성에 따라 오버 혹은 언더 샘플링 방식을 선택한다. 일반적으로 실무에서는 많은 데이터 확보가 우선이므로 오버샘플링 기법을 대부분 적용한다. 대표적으로 소수 범주집단의 데이터를 반복적으로 복사하여 데이터를 생성하는 오버샘플링 기법인 SMOTE(synthetic minority oversampling technique)를 적용하여 예측한다.

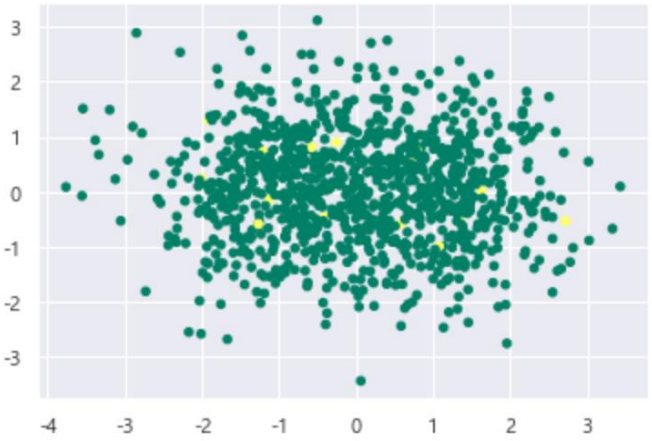
불균형 데이터를 해결하는 방법을 표로 정리하였다(표[]).

해결 방법	설명
데이터 추가 수집	데이터를 추가로 수집확보하여 예측 모형에 투입하는 방법이나 현실성 없음
특성공학 적용	원래 데이터에서 특성공학을 통해 추가적인 파생변수를 찾는 방법
다양한 성능지표	재현율, fbeta_score, G-means, balanced accuracy score, matthews_corrcoef 등
손실함수 조정	손실함수의 허위양성/음성율 구성항목에 가중치를 높여 허위양성/음성을 줄이기
Undersampling	Target의 다수 범주유형의 데이터를 소수 범주유형의 크기로 Undersampling
Oversampling	Target의 소수 범주유형 데이터를 다수 범주유형의 크기로 Oversampling
Combinesampling	Oversampling과 Undersampling을 combine
Ensemble of sampler	앙상블 알고리즘의 데이터 재수집시 Under/Over/combine sampling
데이터 선택	이진 분류일 경우 소수범주에 속하지 않는 특성변수의 일부 데이터를 제거

무작위로 특성 20개의 불균형 데이터(99%, 1%)를 생성하여 특성 2개만으로 시각화한다.

1000개 무작위 데이터 생성

```
from sklearn.datasets import make_classification
X, y = make_classification(n_samples=1000, n_classes=2, weights=[0.99, 0.01],
                           random_state=1)
plt.scatter(X[:, 0], X[:, 1], c=y, s=20, cmap='summer')
```



훈련과 검증데이터 생성

```
X_train, X_test, y_train, y_test = model_selection.train_test_split(X, y, test_size = 0.5,
                                                                    random_state = 2,
stratify=y)
print(f'Dataset: Class 0(음성)= {len(y[y==0])} 개, Class1(양성)={len(y[y==1])} 개')
```

```
Dataset: Class 0(음성)= 985 개, Class1(양성)=15 개
```

예측 모형 분석보고서 결과 양성자료의 정밀도와 재현율은 '0'인 반면 음성은 99% 이상이다.

로직스틱회귀분석 예측 모형 생성

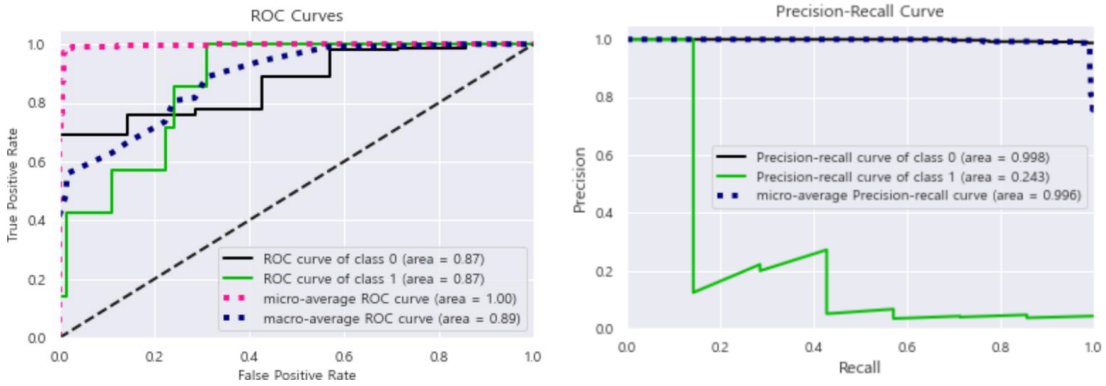
```
from sklearn.linear_model import LogisticRegression
clf = LogisticRegression(solver='lbfgs')
y_pred = clf.fit(X_train, y_train).predict(X_test)
y_probas = clf.fit(X_train, y_train).predict_proba(X_test)
target_names = ['Negative (0)', 'Positive(1)']
print(metrics.classification_report(y_test, y_pred, target_names=target_names))
```

	precision	recall	f1-score	support
Negative (0)	0.99	1.00	0.99	493
Positive(1)	0.00	0.00	0.00	7
accuracy			0.99	500
macro avg	0.49	0.50	0.50	500
weighted avg	0.97	0.99	0.98	500

예측 모형 ROC 곡선은 양성 재현율과 정밀도가 '0'이어도 AUROC가 87%를 나타내고 있다. 불균형데이터인 경우 PR 그래프는 소수 범주 자료(양성, 1%)에 집중하는 반면 ROC curve는 음성과 양성 범주 자료를 모두 고려하기 때문에 PR그래프로 모델을 평가하여야 한다.

불균형데이터의 ROC, PR그래프

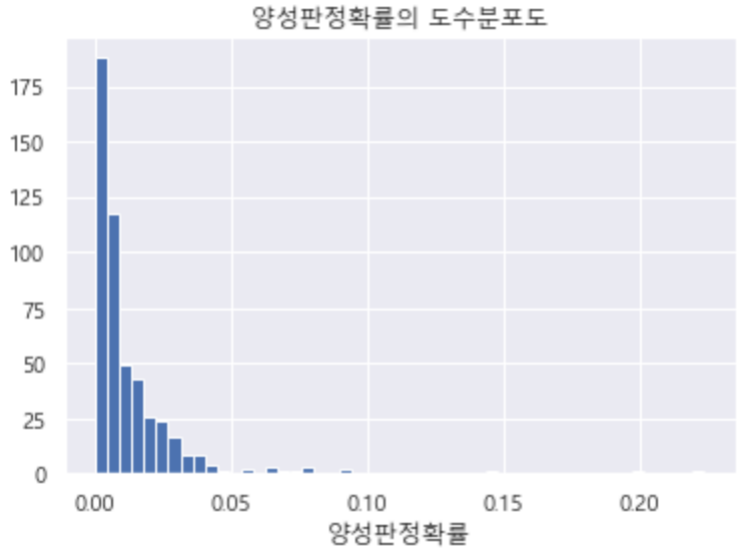
```
import scikitplot as skplt
skplt.metrics.plot_roc(y_test, y_probas)
skplt.metrics.plot_precision_recall(y_test, y_probas)
```



사이킷런 내부 모델의 의사결정 임계값은 '0.5'로 예측 모델의 결과인 500개는 모두 음성으로 판정한 것을 확인할 수 있으며 양성판정 비율은 모두 0.22 이하인 것으로 나타난다.

양성 예측확률의 분포도

```
from collections import Counter
Counter(y_pred)
ax = plt.hist(y_probas[:, 1], bins=50)
plt.title('양성판정확률의 도수분포도'); plt.xlabel('양성판정확률')
```



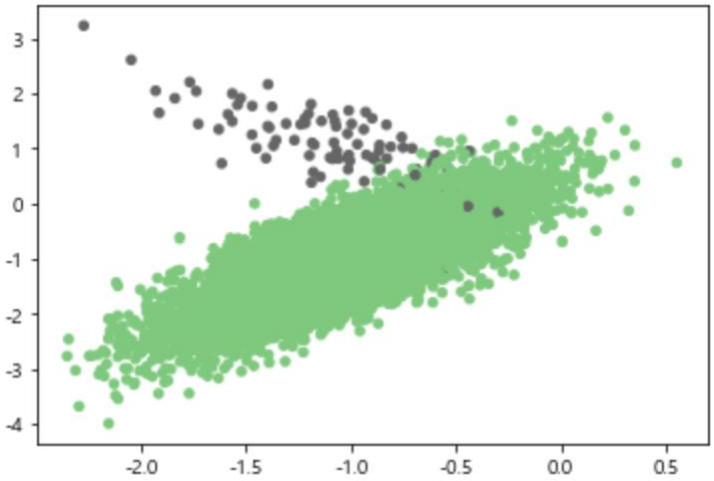
불균형 데이터의 문제점을 해결하기 위한 방법으로 다양한 예측 성능 지표에 따라 연구목적에 맞는 모형을 임의적으로 선택할 수 있다.

■ 다양한 성능 지표 적용

5개 특성을 가진 3개 그룹의 무작위 데이터 10,000개를 생성한다..

무작위 데이터 생성

```
X, y = make_classification(n_samples=10000, n_features=5, n_redundant=3,
                           n_clusters_per_class=1, weights=[0.99], flip_y=0, random_state=4)
plt.scatter(X[:, 0], X[:, 1], c=y, s=20, cmap='Accent')
```



예측모형 분류보고서 확인

```
X_train, X_test, y_train, y_test = model_selection.train_test_split(X, y, test_size=0.5,
                                                    random_state=2, stratify=y)
model = LogisticRegression(solver='lbfgs')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
y_probas = model.predict_proba(X_test)
print(metrics.classification_report(y_test, y_pred, target_names=target_names))
```

	precision	recall	f1-score	support
Negative (0)	1.00	1.00	1.00	4950
Positive(1)	0.97	0.56	0.71	50
accuracy			1.00	5000
macro avg	0.98	0.78	0.85	5000
weighted avg	1.00	1.00	0.99	5000

불균형데이터인 경우에는, 양성과 음성을 모두 고려하여야 한다. 'Geometric Mean' 혹은 G-mean은 45도선과 ROC 곡선과의 거리이다. 최대의 G-mean은 재현율(recall, sensitivity)과 specificity(1-FPR)와의 균형점으로 ROC곡선이 45도선(무작위 예측선)과 가장 거리가 먼 지점의 threshold이다.

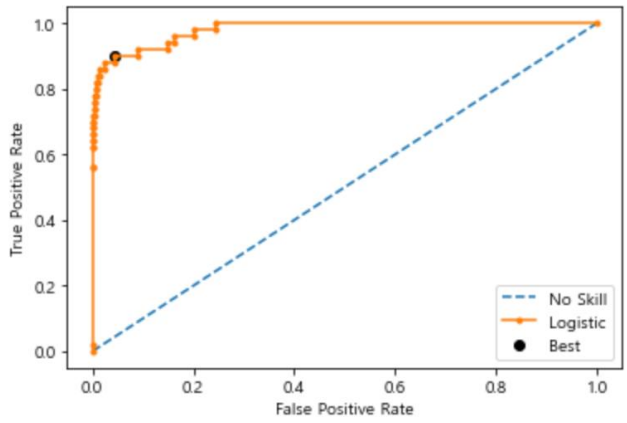
maximum G-mean in ROC curve

```
fpr, tpr, thresholds = metrics.roc_curve(y_test, y_probas[:, 1])
gmeans = np.sqrt(tpr * (1-fpr)) # calculate the g-mean for each threshold
ix = np.argmax(gmeans) # locate the index of the largest g-mean
print(f'Best Threshold={thresholds[ix]}, G-mean={gmeans[ix]}')

plt.plot([0,1], [0,1], linestyle='--', label='No Skill')
```

```
plt.plot(fpr, tpr, marker='.', label='Logistic')
plt.scatter(fpr[ix], tpr[ix], marker='o', color='black', label='Best')
plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate')
plt.legend()
```

```
Best Threshold=0.018570559212865982, G-mean=0.9284395510748128
```



최대의 G-mean을 찾아내는 방법은 Youden's J statistic 값과 동일하다. Youden's J 통계량, J= Sensitivity+Specificity−1= Sensitivity+(1−False Positive Rate)−1= True Positive Rate − False Positive Rate이다.

Youden's J statistics

```
J = tpr - fpr
ix = np.argmax(J)
best_thresh = thresholds[ix]
print(f'Best Threshold= {best_thresh}')
```

```
>>> Best Threshold= 0.018570559212865982
```

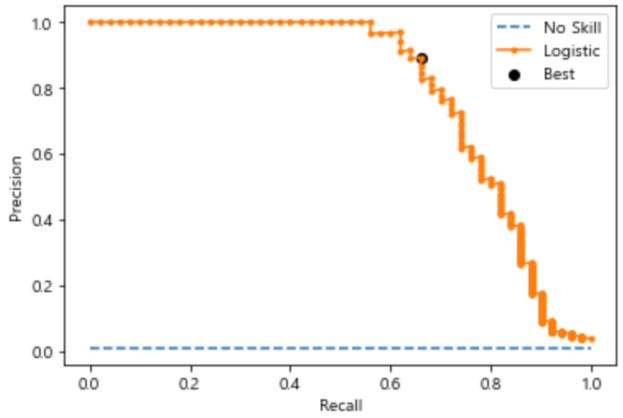
Precision-Recall 곡선은 ROC곡선과는 다르게 소수 범주의 양성 자료를 반영한 예측 성능을 나타낸다. Precision-Recall의 조화평균인 f1-score가 가장 최대인 threshold를 확인한다.

Maximum f1-score in PR curve

```
precision, recall, thresholds= metrics.precision_recall_curve(y_test, y_probas[:,1])
fscore = (2 * precision * recall) / (precision + recall)
ix = np.argmax(fscore) # locate the index of the largest f-measure
print(f'Best Threshold= {thresholds[ix]: .3f}, F-measure={fscore[ix]: .3f}')
no_skill = len(y_test[y_test==1]) / len(y_test)
plt.plot([0,1], [no_skill,no_skill], linestyle='--', label='No Skill')
plt.scatter(recall[ix], precision[ix], marker='o', color='black', label='Best')
plt.plot(recall, precision, marker='.', label='Logistic')
```

```
plt.xlabel('Recall'); plt.ylabel('Precision')
plt.legend()
```

```
Best Threshold=  0.291, F-measure= 0.759
```



일반적으로 threshold에 따른 Precision, Recall, f1-score를 확인하여 연구목적에 적합한 threshold를 선택하게 된다.

Decision threshold curve, recall, precision

```
def to_labels(pos_probs, threshold):
	return (pos_probs >= threshold).astype('int')

probs = y_probas[:, 1] # keep probabilities for the positive outcome only
thresholds = np.arange(0, 1, 0.001)
p, r, thresholds = metrics.precision_recall_curve(y_test, probs)
(pd.DataFrame(np.vstack([p, r]), index=['정밀도', '재현율']).round(2).
 append(pd.DataFrame(thresholds,columns=['분류임계값']).T.round(2))
)
```

	0	1	2	3	4	5	6	7	8	9	...	1257	1258	1259	1260	1261	1262	1263	1264	1265	1266
정밀도	0.04	0.04	0.04	0.04	0.04	0.04	0.04	0.04	0.04	0.04	...	1.00	1.00	1.00	1.00	1.0	1.00	1.00	1.00	1.00	1.0
재현율	1.00	0.98	0.98	0.98	0.98	0.98	0.98	0.98	0.98	0.98	...	0.18	0.16	0.14	0.12	0.1	0.08	0.06	0.04	0.02	0.0
분류임계값	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	...	0.98	0.98	0.99	0.99	1.0	1.00	1.00	1.00	1.00	NaN

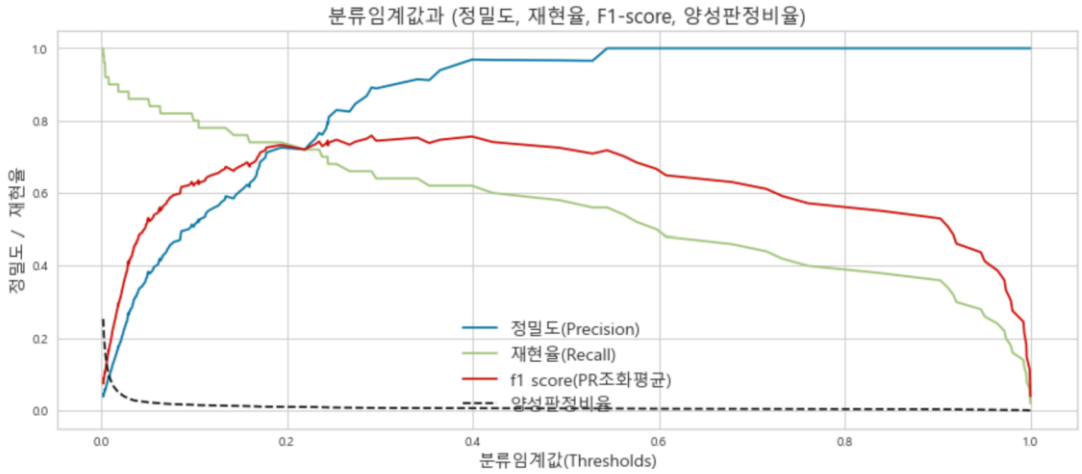
3 rows × 1267 columns

분류임계값(thereshold)과 Precision, Recall, f1-score, 양성판정비율을 시각화한다.

Decision threshold curve

```
scores = [np.sum(to_labels(probs, t))/len(probs) for t in thresholds]
def plot_precision_recall_vs_threshold(precisions, recalls, thresholds):
    plt.rcParams["figure.figsize"] = (15,6)
    plt.title('분류임계값과 (정밀도, 재현율, F1-score, 양성판정비율)', fontsize='16')
    plt.plot(thresholds, precisions[:-1], 'b-', label='정밀도(Precision)') # 판사
    plt.plot(thresholds, recalls[:-1], 'g-', label='재현율(Recall)') # 검사
    plt.plot(thresholds, (2*precisions[:-1]*recalls[:-1])/(precisions[:-1] +recalls[:-1]),
             'r-', label='f1 score(PR조화평균)') # 변호사
    plt.plot(thresholds, scores, 'k--', label='양성판정비율') # q-ratio
    plt.ylabel('정밀도 /  재현율', fontsize='14')
    plt.xlabel('분류임계값(Thresholds)', fontsize='14')
    plt.legend(loc='best', fontsize='14')

plot_precision_recall_vs_threshold(p, r, thresholds)
```



정확도, 재현율, 정밀도 등 여러 가지 성능 지표에 따라 예측 모형의 파라미터는 달라지게 되며 이에 따라 분석보고서의 결과도 달라지게 된다. 재현율과 정밀도를 가중 조화평균한 F-beta score(fbeta_score)는 beta가 '1'보다 크면 재현율에 더욱 비중을 두게 되며 f1-score는 beta가 '1'인 경우로 동일한 비중으로 조화평균한 값이다.

다양한 성능 지표로 불균형데이터 예측 모형 판정

```
ftwo_scorer = metrics.make_scorer(metrics.fbeta_score, beta=2)
scoring = ['accuracy', 'precision', 'recall', 'f1', ftwo_scorer]

params = {'C': np.logspace(-4, 4, 20), 'penalty': ['l1', 'l2']}
for score in scoring:
    grid = model_selection.RandomizedSearchCV(model, param_distributions=params, cv=5,
scoring= score)
    print(f'예측 성능 지표= {score}')
```

```
    print('-'*70)
    y_predict = grid.fit(X_train, y_train).predict(X_test)
    print(metrics.classification_report(y_test, y_predict))
    print('-'*70)
```

예측 성능 지표= accuracy

	precision	recall	f1-score	support
0	1.00	1.00	1.00	4950
1	0.97	0.60	0.74	50
accuracy			1.00	5000
macro avg	0.98	0.80	0.87	5000
weighted avg	1.00	1.00	1.00	5000

예측 성능 지표= f1

	precision	recall	f1-score	support
0	1.00	1.00	1.00	4950
1	0.97	0.60	0.74	50
accuracy			1.00	5000
macro avg	0.98	0.80	0.87	5000
weighted avg	1.00	1.00	1.00	5000

예측 성능 지표= precision

	precision	recall	f1-score	support
0	1.00	1.00	1.00	4950
1	0.97	0.58	0.72	50
accuracy			1.00	5000
macro avg	0.98	0.79	0.86	5000
weighted avg	1.00	1.00	1.00	5000

예측 성능 지표= recall_macro

	precision	recall	f1-score	support
0	1.00	1.00	1.00	4950
1	0.97	0.62	0.76	50
accuracy			1.00	5000
macro avg	0.98	0.81	0.88	5000
weighted avg	1.00	1.00	1.00	5000

예측 성능 지표= recall

	precision	recall	f1-score	support
0	1.00	1.00	1.00	4950
1	0.97	0.60	0.74	50
accuracy			1.00	5000
macro avg	0.98	0.80	0.87	5000
weighted avg	1.00	1.00	1.00	5000

예측 성능 지표= make_scorer(fbeta_score, beta=2)

	precision	recall	f1-score	support
0	1.00	1.00	1.00	4950
1	0.97	0.60	0.74	50
accuracy			1.00	5000
macro avg	0.98	0.80	0.87	5000
weighted avg	1.00	1.00	1.00	5000

복수의 성능지표를 동시에 확인할 경우에는 scores를 make_scorer와 같이 사전형으로 구성하여 확인하면 성능지표에 따라 최적 예측 모형의 파라미터의 차이점을 확인한다.

복수의 성능지표(Multiple metrics)

```
from sklearn.metrics import make_scorer, recall_score
from sklearn.metrics import accuracy_score, precision_score, confusion_matrix
scorers = {
    'precision_score': make_scorer(precision_score),
    'recall_score': make_scorer(recall_score),
    'accuracy_score': make_scorer(accuracy_score)
}
for key, _ in scorers.items():
    print(key)
    grid = model_selection.RandomizedSearchCV(model, param_distributions=params, cv=5,
                                              scoring= scorers,
                                              refit= key)
    grid.fit(X_train, y_train)
```

grid.best_estimator_

```
>>> LogisticRegression(C=1.623776739, class_weight={0:0.01, 1:0.99})
>>> LogisticRegression(C=0.033598182, class_weight={0:0.01, 1:0.99})
>>> LogisticRegression(C=0.012742749, class_weight={0:0.01, 1:0.99})
```

복수의 성능지표별 교차검증과 무작위 서치한 과정은 cv_results로 확인할 수있다.

복수의 성능지표의 교첨검증 무작위 서치결과 확인

```
grid.cv_results_.keys()
results= pd.DataFrame(grid.cv_results_)[['params','rank_test_precision_score',
                                         'rank_test_recall_score','rank_test_accuracy_score']]
results
```

```
dict_keys(['mean_fit_time', 'std_fit_time', 'mean_score_time', 'std_score_time', 'param_penalty', 'param_C', 'params', 'split0_test_precision_score', 'split1_test_precision_score', 'split2_test_precision_score', 'split3_test_precision_score', 'split4_test_precision_score', 'mean_test_precision_score', 'std_test_precision_score', 'rank_test_precision_score', 'split0_test_recall_score', 'split1_test_recall_score', 'split2_test_recall_score', 'split3_test_recall_score', 'split4_test_recall_score', 'mean_test_recall_score', 'std_test_recall_score', 'rank_test_recall_score', 'split0_test_accuracy_score', 'split1_test_accuracy_score', 'split2_test_accuracy_score', 'split3_test_accuracy_score', 'split4_test_accuracy_score', 'mean_test_accuracy_score', 'std_test_accuracy_score', 'rank_test_accuracy_score'])
```

	params	rank_test_precision_score	rank_test_recall_score	rank_test_accuracy_score
0	{'penalty': 'l1', 'C': 0.012742749857031334}	7	7	7
1	{'penalty': 'l1', 'C': 11.288378916846883}	8	8	8
2	{'penalty': 'l1', 'C': 0.00483293023857175 2}	9	9	9
3	{'penalty': 'l2', 'C': 0.615848211066026}	4	1	6
4	{'penalty': 'l2', 'C': 0.08858667904100823}	2	1	2
5	{'penalty': 'l2', 'C': 78.47599703514607}	5	1	3
6	{'penalty': 'l2', 'C': 0.00483293023857175 2}	1	6	1
7	{'penalty': 'l2', 'C': 4.281332398719396}	3	1	5
8	{'penalty': 'l1', 'C': 0.0006951927961775605}	10	10	10
9	{'penalty': 'l2', 'C': 10000.0}	5	1	3

■ 손실함수 조정

불균형 데이터의 문제점을 해결하기 위한 방법으로 손실 함수(loss function)를 조정하여 예측 성능을 향상시킬 수 있다. 손실함수는 허위양성(false positive)과 허위음성(false negative)로 구성된다. 소수 범주의 양성자료에 대한 오류(error)에 패널티를 부여하여 손실함수를 조정하는 것이다. 로지스틱회귀 모형은 내부적으로 손실 함수(Loss function)는 다음과 같이 cross entropy형태의 함수이다.

$$Loss\ function = \text{허위양성} + \text{허위음성} = y \times log^{p} + (1-y) \times log^{(1-p)}$$

소수 범주의 예측 성능을 개선하려면 허위양성에 가중치를 부여하여 소수 범주 양성의 예측에 비중을 높인다.

양성과 음성의 비중을 가중치로 하여 서로 교환하여(class_weight='balanced') 예측한다. 손실함수를 99%중*허위양성 + 1%*허위음성으로 손실함수를 조정하여 예측한다.

손실함수 조정 불균형데이터 예측

```
model = LogisticRegression(solver='lbfgs', class_weight='balanced')
params = {'C': np.logspace(-4, 4, 20), 'penalty': ['l1', 'l2']}
grid = model_selection.RandomizedSearchCV(model, params, cv=5, scoring= 'recall')
y_predict = grid.fit(X_train, y_train).predict(X_test)
print(metrics.classification_report(y_test, y_predict))
```

```
              precision    recall  f1-score   support

           0       1.00      0.92      0.96      4950
           1       0.11      0.92      0.20        50

    accuracy                           0.92      5000
   macro avg       0.55      0.92      0.58      5000
weighted avg       0.99      0.92      0.95      5000
```

파라미터를 class_weight= {0: minor, 1: major})로 하면 동일한 결과이며 허위양성과 음성의 패널티를 다르게 부여하여 예측할 수도 있다.

손실함수 조정 예측

```
minor = np.sum(y)/len(y); major = 1- minor
model = LogisticRegression(solver='lbfgs', class_weight= {0: minor, 1: major})
params = {'C': np.logspace(-4, 4, 20), 'penalty': ['l1', 'l2']}
grid = model_selection.RandomizedSearchCV(model, params, cv=5, scoring= 'recall')
y_predict = grid.fit(X_train, y_train).predict(X_test)
print(metrics.classification_report(y_test, y_predict))
```

	precision	recall	f1-score	support
0	1.00	0.92	0.96	4950
1	0.11	0.92	0.20	50
accuracy			0.92	5000
macro avg	0.55	0.92	0.58	5000
weighted avg	0.99	0.92	0.95	5000

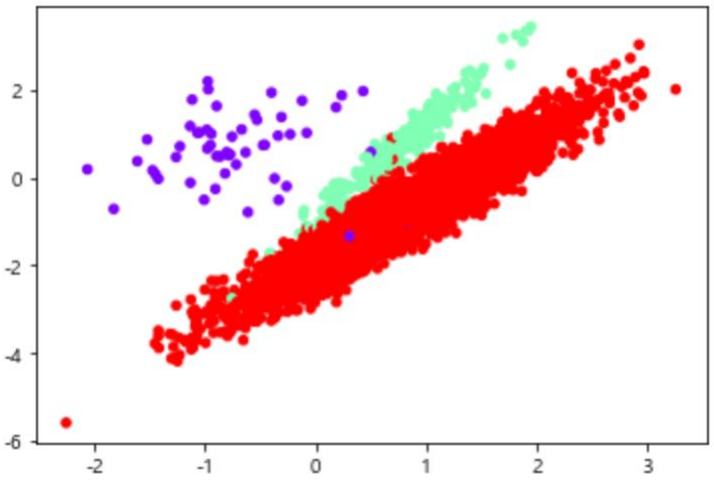
■ Under-Over-Combine-ensemble classifier sampling

불균형 데이터의 문제점을 해결하기 위한 다른 방법으로 데이터를 Oversampling, Undersampling, Over/Undersampling, 앙상블알고리즘의 bootstrapping 시 데이터 조정 방안 등이 있다. 데이터를 조정하여 불균형데이터 예측 성능을 높이기 위한 패키지의 자세한 내용과 유저가이드는 'imbalanced-learn.org/'를 참고하기 바란다.

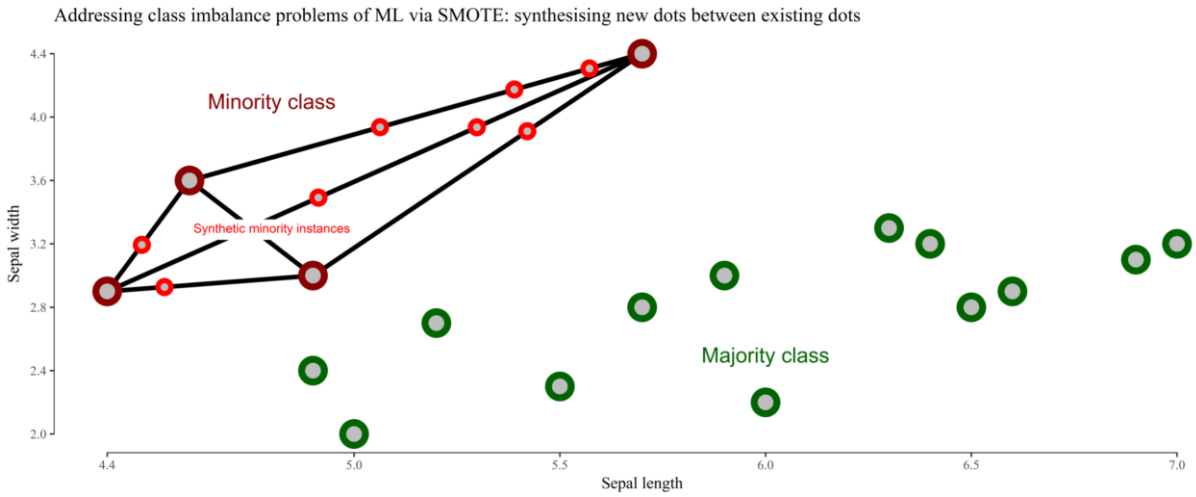
3개 class를 가진 데이터를 임의로 생성하여 시각화한다.

무작위 불균형 데이터 생성(3개 class)

```
from sklearn.datasets import make_classification
X, y = make_classification(n_samples=5000, n_features=2, n_informative=2,
                           n_redundant=0, n_repeated=0, n_classes=3,
                           n_clusters_per_class=1,
                           weights=[0.01, 0.05, 0.94],
                           class_sep=0.8, random_state=0)
plt.scatter(X[:, 0], X[:, 1], c=y, s=20, cmap='rainbow')
```



원래의 불균형데이터를 SMOTE(SMOTE(Synthetic Minority Over-sampling Technique)를 적용하여 3개 범주유형의 데이터 크기를 확인한다. SMOTE는 인스턴스간에 최근접 이웃(KNN)을 이용하여 새로운 가상의 데이터를 생성하는 Oversampling 기법이다.(그림[]). 인스턴스간의 거리를 기준으로 하는 KNN알고리즘은 고차원에서의 거리는 의미가 없기 때문에 SMOTE 알고리즘도 고차원의 데이터에는 적합하지 않다.



출처: youtube.com/watch?v=Qkcce86nqFQ

SMOTE 적용

```
from imblearn.over_sampling import SMOTE
from collections import Counter
print(f'normal data distribution: {Counter(y)}')
X_smote, y_smote = SMOTE().fit_resample(X,y)
print(f'SMOTE data distribution: {Counter(y_smote)}')
```

```
     normal data distribution: Counter({2: 4674, 1: 262, 0: 64})
>>>  SMOTE data distribution: Counter({2: 4674, 1: 4674, 0: 4674})
```

원래 불균형데이터로 LinearSVC 알고리즘을 적용하여 정확도(accuracy)를 확인한다.

원래 데이터로 예측

```
from sklearn import model_selection, metrics
from imblearn.pipeline import make_pipeline
from sklearn.svm import LinearSVC
clf = LinearSVC()
kf = model_selection.KFold(n_splits=3, shuffle=True, random_state=11)
model_selection.cross_validate(clf, X, y, scoring='recall_macro', cv=kf)
model_selection.cross_val_score(clf, X, y, scoring='recall_macro', cv=kf)
```

```
     {'fit_time': array([0.09859943, 0.09154582, 0.0740025 ]),
      'score_time': array([0.00331712, 0.0030005 , 0.0029974 ]),
      'test_score': array([0.69340238, 0.56378601, 0.59348149])}

>>>  array([0.69340238, 0.56378601, 0.59348149])
```

소수 범주유형의 데이터를 반복해서 넣는 것(replacement)으로 손실함수의 가중치를 증가시키는 것과 비슷한 Random Over Sampling, 소수 범주에서 다수 범주와 가장 가까운

k개의 소수 범주 데이터 중 무작위로 선택하여 가상의 데이터를 만드는 ADASYN(Adaptive Synthetic Sampling) 알고리즘으로 예측하여 모형의 성능이 높아지는 것을 확인한다.

Oversampling

```
from imblearn.over_sampling import *
oversamplers = [RandomOverSampler(random_state=0),
                ADASYN(random_state=0), SMOTE(random_state=42)]
for sampler in oversamplers :
    imba_pipeline= make_pipeline(sampler, clf)
    model_selection.cross_val_score(imba_pipeline, X, y, scoring='recall_macro', cv=kf)
     array([0.8378122 , 0.82726718, 0.82383264])

     array([0.53418115, 0.5825033 , 0.71177491])

>>>  array([0.84221685, 0.83272696, 0.82319038])
```

다수와 소수범주 경계선에 있는 데이터를.오버셈플링하는 방법인 BorderlineSMOTE와 SVM알고리즘을 이용한 오버샘플링 방법인 SVMSMOTE 알고리즘을 사용하면 예측 성능이 높아진다.

Borderline SMOTE

```
borderline_smote = [BorderlineSMOTE(random_state=13), SVMSMOTE(random_state=11)]
for sampler in borderline_smote :
    imba_pipeline= make_pipeline(sampler, clf)
    model_selection.cross_val_score(imba_pipeline, X, y, scoring='recall_macro', cv=kf)
     array([0.82447722, 0.86502517, 0.87378737])

>>>  array([0.8946434 , 0.91022688, 0.88909526])
```

무작위로 데이터를 없애는 단순 샘플링인 Ramdom Under-Sampler, 범주 유형이 다른 인스턴스가 아주 가까이 붙어있는 토멕링크를 찾아 다수 클래스에 속하는 데이터를 제외하는 방법인 Tomek's link in, 토맥링크 중 다수 범주유형을 제외한 나머지 범주중에서도 서로 붙어있는 다수 클래스 인스턴스를 제외하는 One Sided Selection의 알고리즘으로 예측한다.

Undersampling

```
from imblearn.under_sampling import *
undersamplers = [RandomUnderSampler(), TomekLinks(), OneSidedSelection()]
for sampler in undersamplers :
    imba_pipeline= make_pipeline(sampler, clf)
    model_selection.cross_val_score(imba_pipeline, X, y, scoring='recall_macro', cv=kf)
     array([0.82407534, 0.81562667, 0.82083542])

     array([0.71716125, 0.57201646, 0.61246442])

>>>  array([0.73879245, 0.78843996, 0.79414838])
```

SMOTE+ENN은 SMOTE와 Undersampling방법인 ENN(Edited Nearest Neighbours)을 혼합한 것이고 SMOTE+Tomek 방법은 SMOTE와 토멕링크 방법을 섞은 것이다. 혼합 방법이 예측 모형의 성능이 가장 높은 것을 확인할 수 있다.

Combinesampling

```
from imblearn.combine import *
combinesamplers = [SMOTEENN(random_state=0), SMOTETomek(random_state=4)]
for sampler in combinesamplers :
    imba_pipeline= make_pipeline(sampler, clf)
    model_selection.cross_val_score(imba_pipeline, X, y, scoring='recall_macro', cv=kf)
     array([0.86445475, 0.86012321, 0.83558527])

>>>  array([0.84414859, 0.83571076, 0.82468899])
```

앙상블 알고리즘에서 bootstraping으로 resampling을 under/oversampling하는 방법이다.

앙상블 알고리즘

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from imblearn.ensemble import BalancedRandomForestClassifier
from imblearn.ensemble import RUSBoostClassifier

bag = BaggingClassifier(base_estimator=DecisionTreeClassifier(), random_state=0)
brf = BalancedRandomForestClassifier(n_estimators=100, random_state=0)
rusboost = RUSBoostClassifier(n_estimators=200, algorithm='SAMME.R',
                              random_state=0)

for classifier in [bag, brf, rusboost] :
    model_selection.cross_val_score(classifier, X, y, scoring='recall_macro', cv=kf)
```

```
      array([0.86744119, 0.88333391, 0.85821205])
      array([0.88380873, 0.88856932, 0.85341355])
>>>   array([0.57121883, 0.50233915, 0.5361327 ])
```

■ Data downsizing

불균형데이터의 예측 성능을 높이는 마지막 방법은 원래 데이터에서 변수의 특정 값과 관련되어 항상 다수 범주 유형인 자료를 제거하여 다수 범주 유형의 데이터를 줄이는 방법이다. 카드회사의 표준화된 데이터(28 만여 개)의 10%의 자료를 확보한다. 소수범주 target 의 class 는 0.1861%에 불과한 극심한 불균형데이터이다.

Credit Card Fraud Detection 데이터 확보

```
from sklearn.datasets import fetch_openml
data = fetch_openml(name='CreditCardFraudDetection')
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target
df['log_Amount'] = np.log(1+df['Amount'])
data = df.sample(frac=0.1, random_state=135).copy()
data.head()
data.shape
data.target.value_counts(normalize=True)
```

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V22
206709	136330.0	2.108286	-0.020359	-2.234273	-0.124080	0.559843	-1.315913	0.631887	-0.385490	0.150628	...	0.519705
164567	116819.0	2.080143	-0.075408	-1.359381	0.261263	0.232201	-0.797886	0.233487	-0.330165	0.493154	...	-0.784150
16546	27917.0	1.152650	0.204938	0.441832	1.211595	-0.254895	-0.383380	-0.004286	-0.019590	0.085094	...	0.209828
152705	97443.0	-0.389692	0.410148	0.616187	-0.686644	1.040312	-0.322199	0.523625	-0.037453	1.319237	...	0.685267
10646	17920.0	-1.306986	0.183306	3.164803	0.810135	1.529576	2.388322	0.551279	-0.054401	1.675481	...	0.556523

5 rows × 32 columns

```
(28481, 32)

0.0    0.998139
1.0    0.001861
Name: target, dtype: float64
```

거래금액(log-Amount)과 3개 변수로 5차 교차검증으로 10회 재현율을 생성한다.

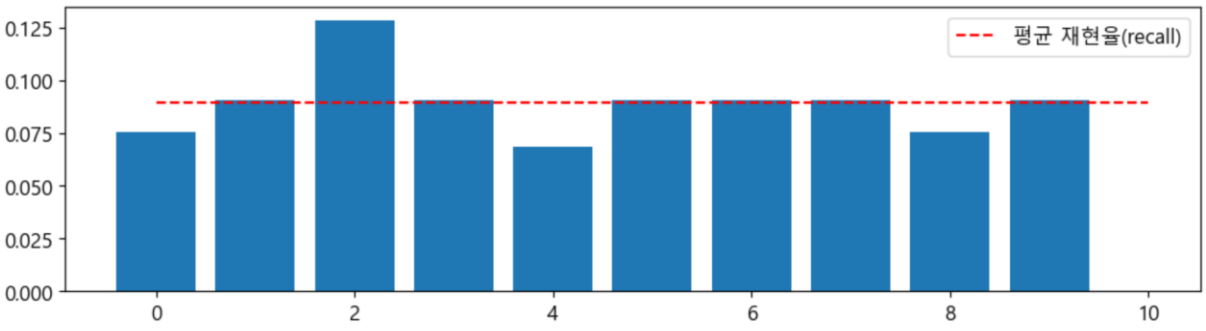
학습과 예측

```
X= data[['log_Amount','V1', 'V3', 'V5']]
y= data['target']
kf = model_selection.KFold(n_splits=5, shuffle=True, random_state=11)
recall_score = []
for seed in np.random.choice(100, 10):
    np.random.seed(seed)
    recall_score.append(model_selection.cross_val_score(clf, X, y,   #
                                                        scoring='recall', cv=kf).mean())
```

5차 교차검증 10회의 재현율과 평균 재현율을 확인한다. 무작위 10회는 매 번 달라진다.

10회 재현율과 평균 재현율 시각화

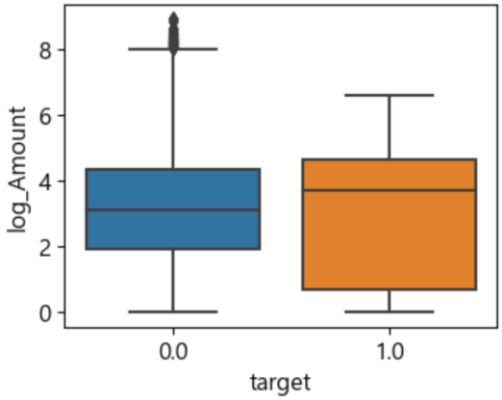
```
plt.rcParams['font.family']='Malgun Gothic'
plt.figure(figsize=(12,3))
plt.bar(np.arange(10), recall_score)
mean = np.mean(recall_score)
plt.plot([0, 10], [mean, mean], 'r--', label='평균 재현율(recall)'); plt.legend()
```



양성과 음성을 기준으로 거래금액(log-Amount)의 Box-plot을 확인한다.

Box-plot 확인

```
import seaborn as sns
sns.boxplot(x= 'target', y= 'log_Amount', data= data)
```



양성인 경우 거래금액(log-Amount)의 최대값(6.585)보다 큰 거래금액(log-Amount)은 모두 음성인 데이터를 제외하여 데이터를 재구성하며 데이터의 크기가 27,960개로 줄어든다. .

불필요한 데이터 제거

```
max = data[data['target'] == 1]['log_Amount'].max()
data.loc[data['log_Amount'] > max]['target'].value_counts()
```

```
print(f'log_Amount 변수값이 {max:.3f}이상이면 항상 적합')
data = data.loc[data['log_Amount'] <= max]
data.shape
```

```
0.0    521
Name: target, dtype: int64

log_Amount 변수값이 6.585이상이면 항상 적합

>>> (27960, 32)
```

새로운 데이터로 거래금액(log-Amount)과 3개 변수로 5차 교차검증으로 10회 재현율을 생성하면 원래 데이터 보다 재현율이 높아진 것을 확인 할 수 있다.

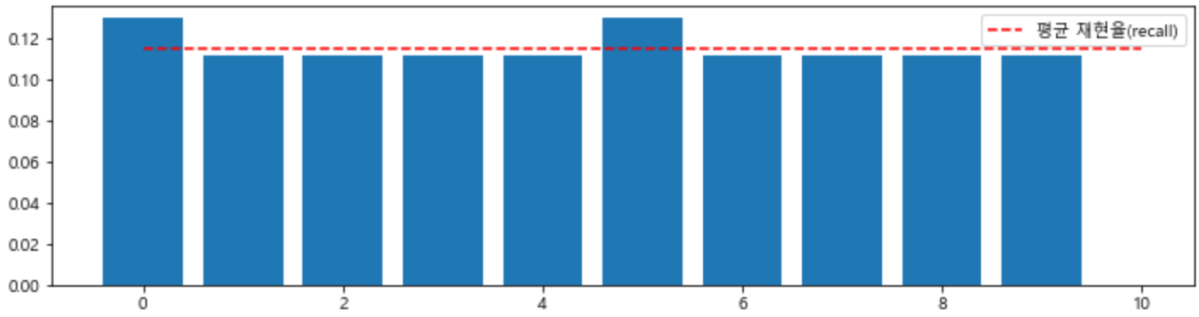
학습과 예측

```
data.target.value_counts(normalize=True)
X= data[['log_Amount','V1', 'V3', 'V5']]
y= data['target']
recall_score = []
for seed in np.random.choice(100, 10):
    np.random.seed(seed)
    recall_score.append(model_selection.cross_val_score(clf, X, y,  #
                                                        scoring='recall', cv=kf).mean())
```

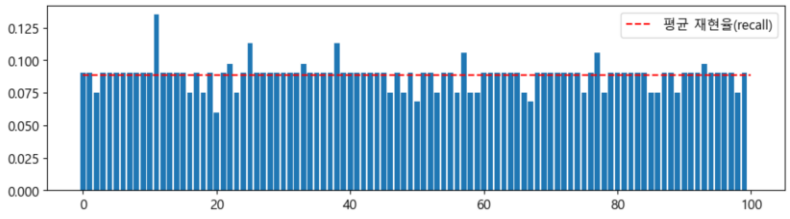
```
0.0    0.998104
1.0    0.001896
>>> Name: target, dtype: float64
```

10회 재현율과 평균 재현율 시각화

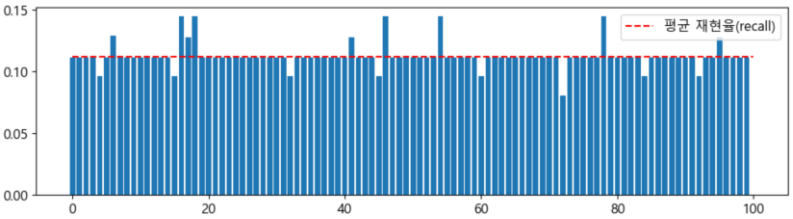
```
plt.figure(figsize=(12,3))
plt.bar(np.arange(10), recall_score)
mean = np.mean(recall_score)
plt.plot([0, 10], [mean, mean], 'r--', label='평균 재현율(recall)')
plt.legend()
```



무작위 추출 난수를 np.random.choice(1000, 100)하여 원래데이터와 불필요한 데이터를 제거한 재현율 simulation 100회를 비교해 보아도 같은 결과를 보인다.



원래 데이터 100 회 재현율 분포



축소된 데이터 100 회 재현율 분포

특성 변수가 범주형일 경우에는 데이터를 축소하는 기준 값을 어떻게 탐지할 수 있는지 살펴보자. Kaggle(/shakedzy/alone-in-the-woods-using-theil-s-u-for-survival)의 예제를 참고하였다. 범주형과 실수형 변수 상관없이 상관계수를 구하는 함수를 만든다.

사용자 정의함수 생성

```
import math
import scipy.stats as ss
from collections import Counter
def conditional_entropy(x,y):
    # entropy of x given y
    y_counter = Counter(y)
    xy_counter = Counter(list(zip(x,y)))
    total_occurrences = sum(y_counter.values())
    entropy = 0
    for xy in xy_counter.keys():
        p_xy = xy_counter[xy] / total_occurrences
        p_y = y_counter[xy[1]] / total_occurrences
        entropy += p_xy * math.log(p_y/p_xy)
    return entropy
def theil_u(x,y):
    s_xy = conditional_entropy(x,y)
    x_counter = Counter(x)
    total_occurrences = sum(x_counter.values())
    p_x = list(map(lambda n: n/total_occurrences, x_counter.values()))
    s_x = ss.entropy(p_x)
    if s_x == 0:
        return 1
    else:
        return (s_x - s_xy) / s_x
>>>
```

데이터를 확보하여 확인하면 수치형이지만 실제 의미는 모두 범주형(categorical type)이다.

Mushroom data(모두 범주형 특성변수)

```
data = fetch_openml(name='mushroom')
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target
df.head()
```

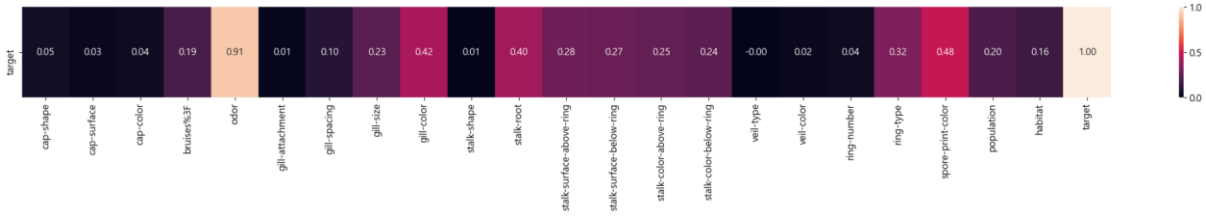
	cap-shape	cap-surface	cap-color	bruises%3F	odor	gill-attachment	gill-spacing	gill-size	gill-color	stalk-shape	...	stalk-color-above-ring	stalk-color-below-ring	veil-type	veil-color	ring-number	ring-type	spore-print-color	population	habitat	target
0	5.0	2.0	4.0	1.0	6.0	2.0	0.0	1.0	4.0	0.0	...	7.0	7.0	0.0	2.0	1.0	5.0	2.0	3.0	5.0	p
1	5.0	2.0	9.0	1.0	0.0	2.0	0.0	0.0	4.0	0.0	...	7.0	7.0	0.0	2.0	1.0	5.0	3.0	2.0	1.0	e
2	0.0	2.0	8.0	1.0	3.0	2.0	0.0	0.0	5.0	0.0	...	7.0	7.0	0.0	2.0	1.0	5.0	3.0	2.0	3.0	e
3	5.0	3.0	8.0	1.0	6.0	2.0	0.0	1.0	5.0	0.0	...	7.0	7.0	0.0	2.0	1.0	5.0	2.0	3.0	5.0	p
4	5.0	2.0	3.0	0.0	5.0	2.0	2.0	0.0	4.0	1.0	...	7.0	7.0	0.0	2.0	1.0	1.0	3.0	0.0	1.0	e

5 rows × 23 columns

모든 변수, 범주형과 실수형과 상관없이 상관계수와 유사하게 상관성을 보여주는 통계량을 계산하는 함수를 사용하여 target과의 상관성을 보여주는 시각화를 구현한다.

모든 변수(범주형, 실수형)의 상관성 분석 시각화

```
columns = df.columns
theilu = pd.DataFrame(index=['target'], columns= columns)
def cat_corr (df):
    for j in range(0,len(columns)):
        u = theil_u(df['target'].tolist(),df[columns[j]].tolist())
        theilu.loc[:,columns[j]] = u
    theilu.fillna(value=np.nan,inplace=True)
    plt.figure(figsize=(30,2))
    sns.heatmap(theilu,annot=True,fmt='.2f')
    plt.show()
cat_corr(df)
```

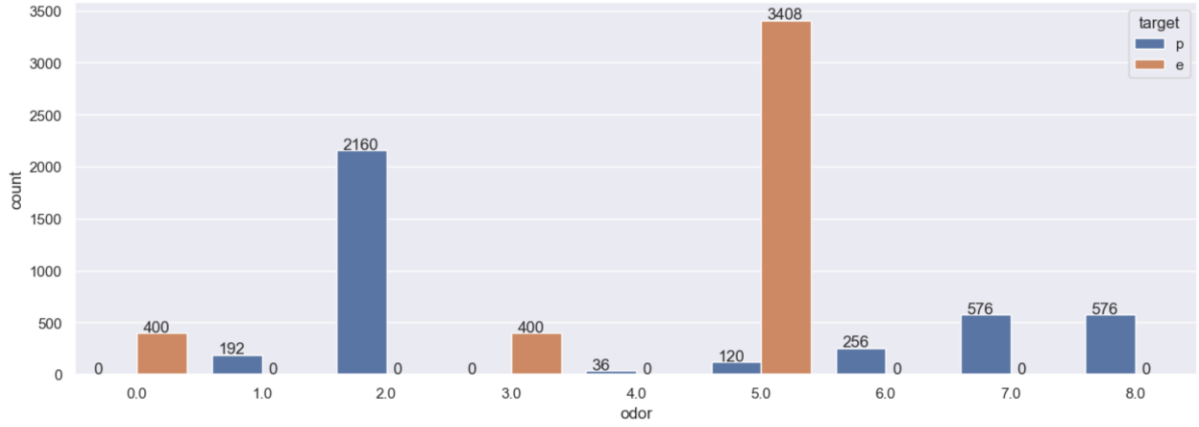


Target과 가장 큰 상관성(0.91)의 'odor'변수를 범주유형별로 양성과 음성의 개수를 시각화하다. 버섯의 냄새(odor)의 유형이 '5.0'인 경우를 제외하고는 다른 유형은 모두 독이 있거나 없고 냄새의 유형 '5.0'인 경우에만 식용과 비식용이 혼재하고 있다.

냄새(odor) 특성의 범주 유형별 양성과 음성 시각화

```
sns.set(rc={'figure.figsize':(15,8)})
ax=sns.countplot(x='odor',hue='target',data=df)
for p in ax.patches:
    patch_height = p.get_height()
    if np.isnan(patch_height):
        patch_height = 0
    ax.annotate('{}'.format(int(patch_height)), (p.get_x()+0.05, patch_height+10))
```

plt.show()



냄새를 맡아보고 냄새가 없는 경우('5.0')이외 냄새가 나는 경우(0~8)는 모두 명백히 식용인지 비식용인지 확인이 가능하다. 버섯의 냄새(odor)의 유형이 '5.0'인 경우를 제외한 데이터는 제거하고 target과의 상관성을 살펴보면 ''spore-print-color'변수가 가장 상관성이 크다.

불필요한 데이터 제거 및 변수의 상관성 분석 시각화

```
no_odor = df[df['odor'].isin([5.0])]
for j in range(0,len(columns)):
    u = theil_u(no_odor['target'].tolist(),no_odor[columns[j]].tolist())
    theilu.loc[:,columns[j]] = u
theilu.fillna(value=np.nan,inplace=True)
plt.figure(figsize=(20,1))
sns.heatmap(theilu,annot=True,fmt='.2f')
plt.show()
```

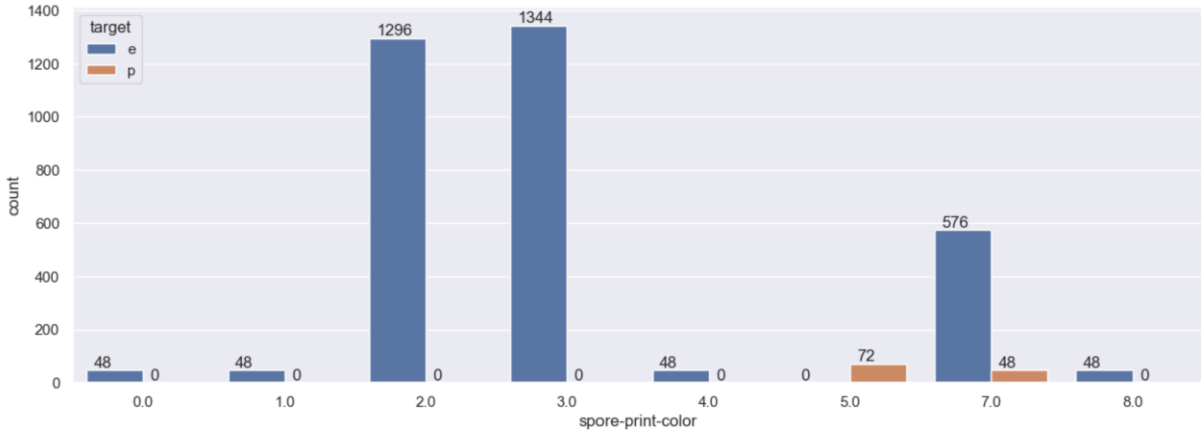


냄새가 나는 데이터(odor=5.0)의 버섯색깔('spore-print-color')변수의 유형별 양성과 음성의 개수 보면 버섯의 색깔 '7.0'(흰색)이 경우만 식용과 비식용이 혼재되어 있다.

버섯색깔(spore-print-color) 특성의 범주 유형별 양성과 음성 시각화

```
sns.set(rc={'figure.figsize':(15,8)})
ax=sns.countplot(x='spore-print-color',hue='target',data=no_odor)
for p in ax.patches:
```

```
        patch_height = p.get_height()
        if np.isnan(patch_height):
            patch_height = 0
        ax.annotate('{}'.format(int(patch_height)), (p.get_x()+0.05, patch_height+10))
plt.show()
```



원래 데이터(8,124개)는 냄새와 색깔 특성변수를 사용하여 624개 92%로 축소되었다. .

불필요한 데이터 제거

```
no_odor_w = no_odor[no_odor['spore-print-color'].isin([7.0])]
df.shape
no_odor_w.shape
(len(df) - len(no_odor_w)) / len(df)
    0.9231905465288035
    (8124, 23)
>>> (624, 23)
```

원래데이터를 예측 모형으로 투입하여 재현율을 확인하면 모두 100%이다.

원래 데이터의 예측 성과지표

```
from sklearn import ensemble
rf = ensemble.RandomForestClassifier(random_state=11)
df['target'].value_counts(normalize=True)
X= df.drop(['target', 'stalk-root'], axis=1)
X.shape
y= df.target.replace({'e':0, 'p':1})
model_selection.cross_val_score(rf, X, y, scoring='recall', cv=kf)
```

```
e    0.517971
p    0.482029
Name: target, dtype: float64

(8124, 17)

>>>  array([1., 1., 1., 1., 1.])
```

불균형데이터 형태가 된 줄어든 데이터로 예측을 하여도 재현율은 모두 100%이다. 원래 데이터가 균형데이터라면 데이터를 축소하여 제현율을 높일 수 있다.

축소된 데이터의 예측 성과지표

```
no_odor_w['target'].value_counts(normalize=True)
X= no_odor_w.drop(['target', 'stalk-root'], axis=1)
X.shape
y= no_odor_w.target.replace({'e':0, 'p':1})
model_selection.cross_val_score(rf, X, y, scoring='recall', cv=kf)
```

```
e    0.923077
p    0.076923
Name: target, dtype: float64

(624, 21)

>>>  array([1., 1., 1., 1., 1.])
```